Monsoon 2025

Distributed Systems

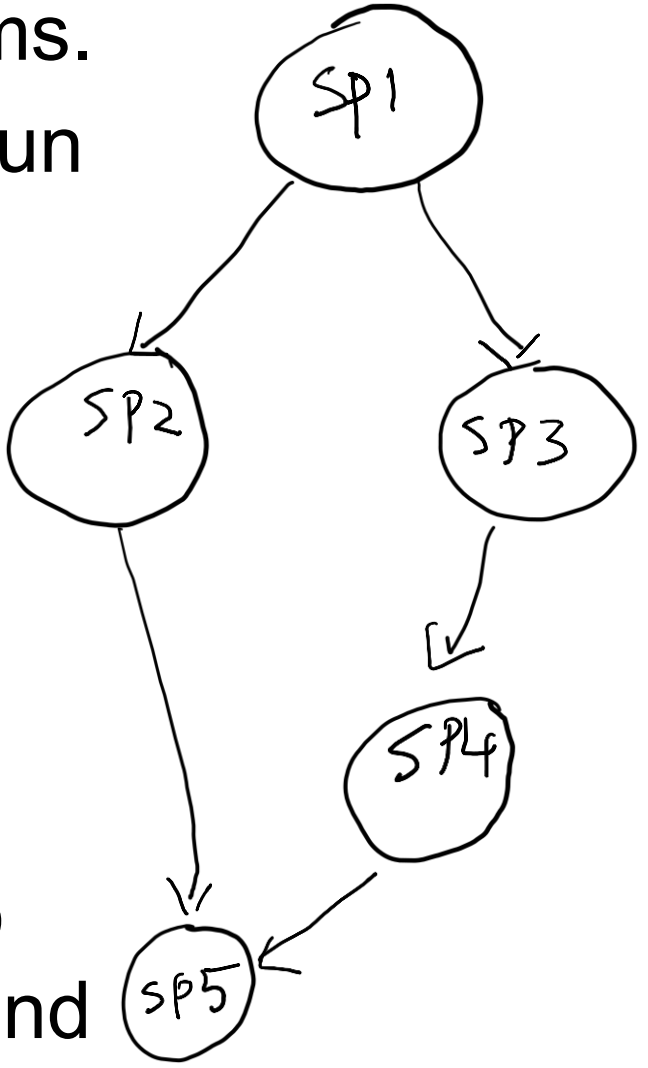
Lecture 13

International Institute of Information Technology

Hyderabad, India

Termination Detection

- Consider a computation consisting of several distributed subprograms.
- All these subprograms may not run on the same system.
- There might exist dependencies across subprograms.
  - A subprogram P has to terminate before a subprogram Q can start execution.
- So, the processor in the system that initiates program Q needs to know if program P has finished and program Q can start execution.



Termination Detection

- Termination detection is an important aspect of distributed computing.
- Simple definition: Check if a distributed computation has Terminated.
- Characterized as follows: A distributed computation is globally terminated if every process is locally terminated and there is no message in transit between any processes.

Termination Detection

- A non-trivial task since no process has complete knowledge of the global state, and global time does not exist.
- “Locally terminated” state is a state in which a process has finished its computation and will not restart any action unless it receives a message.
- In the termination detection problem, a particular process (or all of the processes) must infer when the underlying computation has terminated.

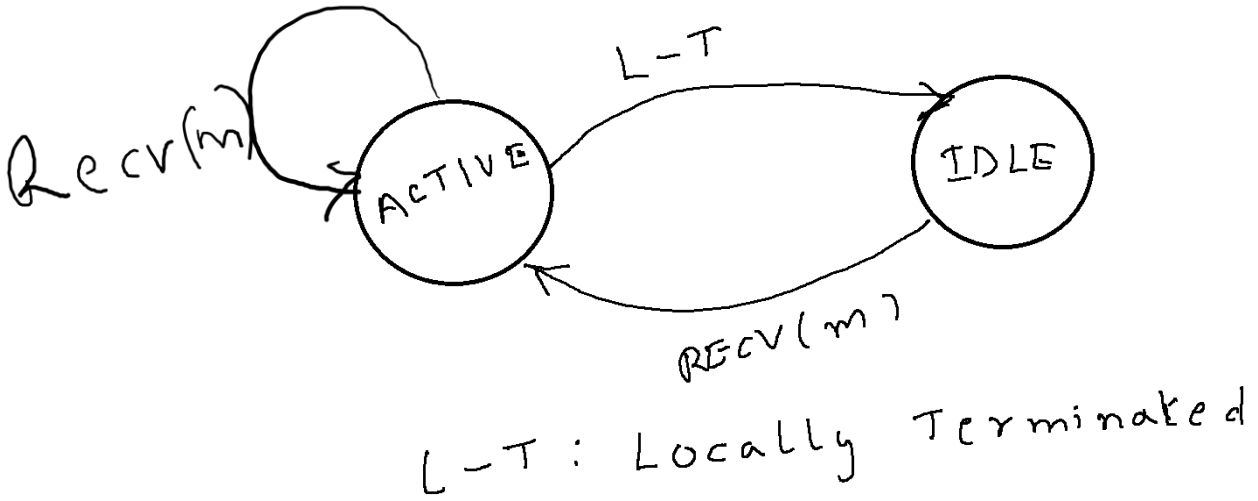
Termination Detection

- A termination detection algorithm is used for this purpose.
- A termination detection (TD) algorithm must ensure the following conditions:
  1. Execution of the TD algorithm cannot indefinitely delay the underlying computation.
  2. The termination detection algorithm must not require addition of new communication channels between processes.

Termination Detection

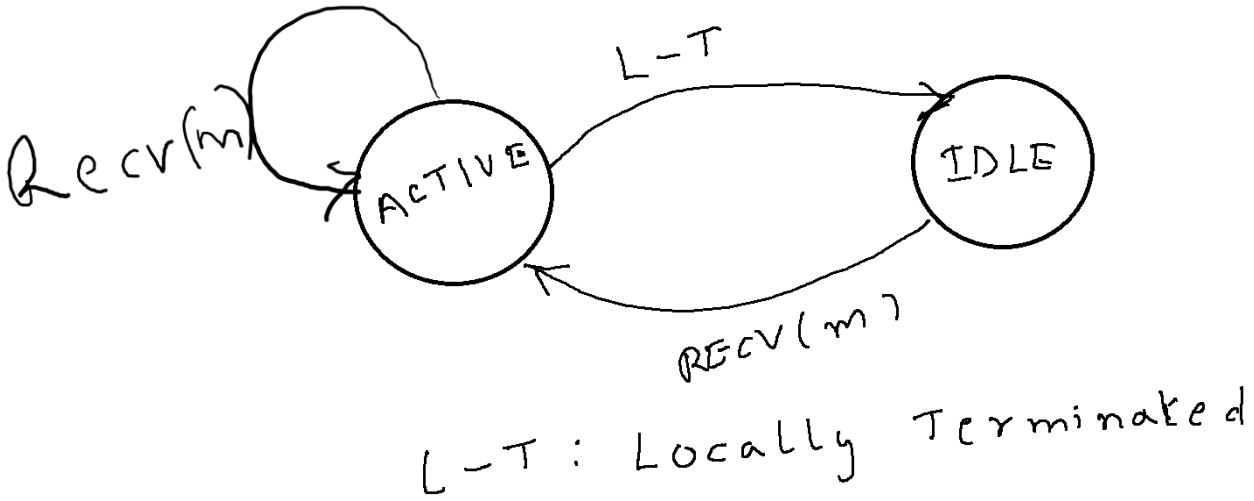
- Most algorithms for termination detection work with two kinds of messages.
- Basic messages: Messages used in the underlying computation
- Control messages: Messages used for the purpose of termination detection.

Assumptions About the System



- At any given time, a process can be in only one of the two states: active, and idle
- Active:
  - Doing local computation
  - An active process can become idle at any time.
- Idle:
  - Finished the execution of its local computation,
  - Can be reactivated only on the receipt of a message from another process.

Assumptions About the System



- An active process can become idle at any time.
- An idle process can become active only on the receipt of a message from another process.
- Only active processes can send messages.
- A message can be received by a process when the process is in either of the two states, i.e., active or idle.
- On the receipt of a message, an idle process becomes active.

Characterization of Termination Detection

- Let $p_i(t)$ denote the state (active or idle) of process p_i at instant t.
- Let $c_{i,j}(t)$ denote the number of messages in transit in the channel at instant t from process p_i to process p_j.
- A distributed computation is said to be terminated at time instant t_0 iff:

$$(\forall i: p_i(t_0) = \text{idle}) \wedge (\forall\ i, j: c_{i,j}(t_0)=0).$$

- Thus, a distributed computation has terminated iff all processes have become idle and there is no message in transit in any channel.

Termination detection Via Distributed Snapshots

- The algorithm assumes that there is a logical bidirectional communication channel between every pair of processes.
- Communication channels are reliable but arbitrary.
- Message delay is arbitrary but finite.
- Use snapshot requests and the snapshots to infer termination.

Termination detection Using Distributed Snapshots

- Main idea:
- When a process goes from active to idle, it issues a request to all other processes to take a local snapshot, and also requests itself to take a local snapshot.
- When a process receives the request, if it agrees that the requester became idle after itself, it grants the request by taking a local snapshot for the request.
- A request is successful if all processes have taken a local snapshot for it.
- The requester or any external agent may collect all the local snapshots of a request.
- If a request is successful, a global snapshot of the request can thus be obtained and the recorded state will indicate termination of the computation.

Termination detection Using Distributed Snapshots

- Main idea:
- When a process goes from active to idle, it issues a request to all other processes to take a local snapshot, and also requests itself to take a local Snapshot.
- When a process receives the request, if it agrees that the requester became idle after itself, it grants the request by taking a local snapshot for the request.
- A request is successful if all processes have taken a local snapshot for it.
- The requester or any external agent may collect all the local snapshots of a request.
- If a request is successful, a global snapshot of the request can thus be obtained and the recorded state will indicate termination of the computation.

More Formal Details

- Each process i maintains a logical clock denoted by x, initialized to zero at the start of the computation.
- A process increments its clock x by one each time it becomes idle.
- B(x): A basic message sent by a process at its logical time x is of the form.
- R(x, i): A control message that requests processes to take local snapshot issued by process i at its logical time x.
- Each process synchronizes its logical clock as per the rules of scalar time.
  - The current time is the maximum of clock values ever received or sent in messages.

More Formal Details

- Every process i also maintains a variable k = $\max_j$ R(x, j)
  - When the process i is idle, (x, k) is the maximum of the values (x, j) on all messages R(x, j) ever received or sent by the process i.

- Logical time, or the tuple (x,j), is compared as follows:
  - (x, j) > (x', j') iff (x > x') or ((x=x') and (j>j'))
  - Example: (4, 5) > (5, 3), (6, 7) > (6, 2)
  - In other words, a tie between times x and x' is broken by the process identification numbers j and j'.
  - Recall the notion of total order on scalar time.

Rules of the Algorithm

- The algorithm is defined by the following four rules.
- (Rule 1): When process i is active, it may send a basic message to process j at any time by doing send a B(x) to j.
- (Rule 2): Upon receiving a B(x'), process i does
  - let x:=max(x,x')+1;
  - if (i is idle) → go active.

- Rule 2 indicates how the processors update their logical clock.

Rules of the Algorithm

- The algorithm is defined by the following four rules.
- (Rule 3): When process i goes idle, it does
  - let x:=x+1;
  - let k := i;
  - send message R(x, k) to all other processes;
  - take a local snapshot for the request by R(x, k).
- Rule 3 states that when a process becomes idle, it issues a broadcast to all other processors requesting a snapshot.

Rules of the Algorithm

- The algorithm is defined by the following four rules.
- (R4): Upon receiving message R(x', k'), process i does
  - If i is active, set x to max(x', x).
  - If (x', k') > (x, k) and (i is idle), set (x, k):= (x', k'); take a local snapshot for the request by R(x', k');
  - If (x', k') ≤ (x, k) and (i is idle), do nothing;
- Rule 4 states that if a process is idle and receives a snapshot request with a larger timestamp, the request is acceded to.
  - For requests with a smaller timestamp, no snapshot is taken.

Rules of the Algorithm

- The algorithm is defined by the following four rules.
- Rule 4 states that if a process is idle and receives a snapshot request with a larger timestamp, the request is acceded to.
  - For requests with a smaller timestamp, no snapshot is taken.
- Intuition:
  - The last process to terminate will have the largest clock value.
    - Why?
  - Therefore, *every* process will take a snapshot for it.

Summary and Discussion

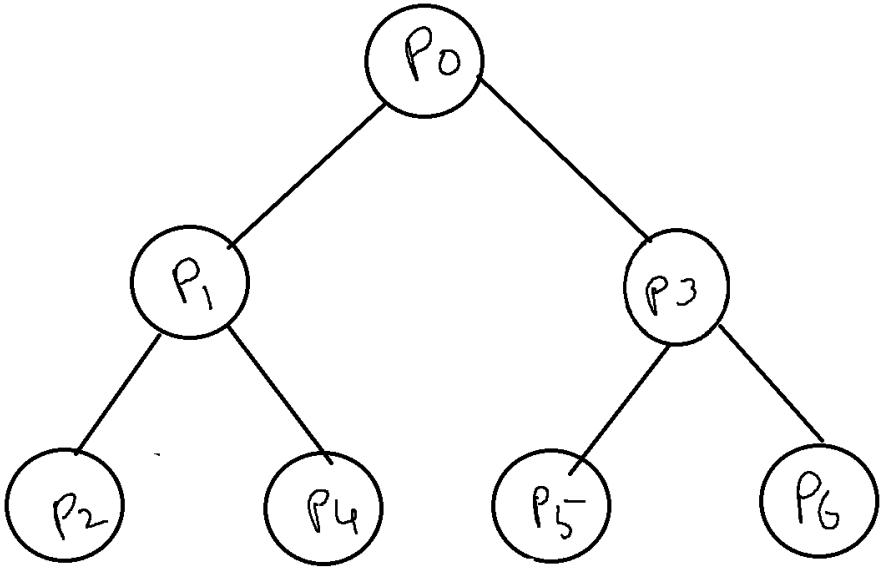
- The algorithm can be quite costly in terms of the messages needed.
- What could happen in the worst case?

Termination Detection via Spanning Trees

- This approach does not rely on obtaining several snapshots.
- The N processors in the distributed system are arranged into a logical spanning tree.
- Process P0 is the root of the spanning tree and is responsible for detecting termination.
- Assume that all communication is synchronous.

Spanning Tree Based Termination Detection

- Process P0 communicates with other processes to determine their states through signals.
- All leaf nodes report to their parents, if they have terminated.
- A parent node will similarly report to its parent when it has completed processing and all of its immediate children have terminated, and so on.
- The root concludes that termination has occurred, if it has terminated and all of its immediate children have also terminated.



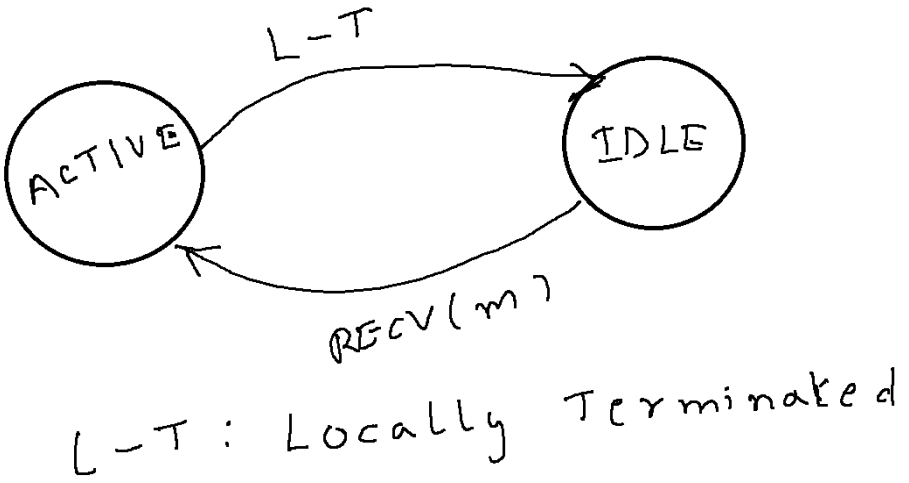
A First Attempt

- Initially, each leaf process is given a token.
- Each leaf process, after it has terminated sends its token to its parent.
- When a parent process terminates and after it has received a token from each of its children, it sends a token to its parent.
- This way, each process indicates to its parent process that the subtree below it has become idle.
- In a similar manner, the tokens get propagated to the root.
- The root of the tree concludes that termination has occurred, after it has become idle and has received a token from each of its children.

A First Attempt

- What could go wrong with the above algorithm?

A First Attempt



- What could go wrong with the above algorithm?
- How to know a process that reported to its parent as idle will never become active again?
- Recall the process state diagram in this context.
  - A message to an idle process can make the process active again!

A First Attempt

- So, even after sending the token to the parent, a processor that is idle may not terminate after all.
- The wrong token may be propagated till P0 which declares termination incorrectly.
- Hence, the root node just because it received a token from a child, cannot conclude that all processes in the child's subtree have terminated.

Fixing the First Attempt

- Need more signals to flow that messages could be in transit while some process declares local termination.
- Plus, the root should not declare termination quickly, but must verify the claim of termination.
- Two waves of signals generated one moving inward and other outward through the spanning tree.
- Initially, a contracting wave of signals, called tokens, moves inward from leaves to the root.
- If this token wave reaches the root without discovering that termination has occurred, the root initiates a second outward wave of repeat signals.
- As this repeat wave reaches leaves, the token wave gradually forms and starts moving inward again, this sequence of events is repeated until the termination is detected.

Fixing the Problem.

- Initially,
  - Each leaf process is provided with a token. The set S is used for book-keeping to know which processes have the token.
  - The set S will be the set of all leaves in the tree.
  - All processes and tokens are colored white.
- When a leaf node terminates, it sends the token it holds to its parent process.
- A parent process will collect the token sent by each of its children.
- After it has received a token from all of its children and after it has terminated, the parent process sends a token to its parent.

Fixing the Problem.

- A process turns black when it sends a message to some other process.
- When a process terminates, if its color is black, it sends a black token to its parent.
- A black process turns white after it has sent a black token to its parent.
- A parent process holding a black token (from one of its children), sends only a black token to its parent, to indicate that a message-passing was involved in its subtree.

Fixing the Problem

- Tokens are propagated to the root in this fashion.
- The root, upon receiving a black token, will know that a process in the tree had sent a message to some other process.
- Hence, it restarts the algorithm by sending a Repeat signal to all its children.

Fixing the Problem

- Each child of the root propagates the Repeat signal to each of its children and so on, until the signal reaches the leaves.
- The leaf nodes restart the algorithm on receiving the Repeat signal.
- The root concludes that termination has occurred, if
  1. it is white,
  2. it is idle, and
  3. it received a white token from each of its children.

Fixing the Problem

- The best case message complexity of the algorithm is O(N), where N is the number of processes in the computation, which occurs when all nodes send all computation messages in the first round.
- The worst case complexity of the algorithm is O(N*M), where M is the number of computation messages exchanged, which occurs when the root has to restart termination detection M times.

Further Improvements

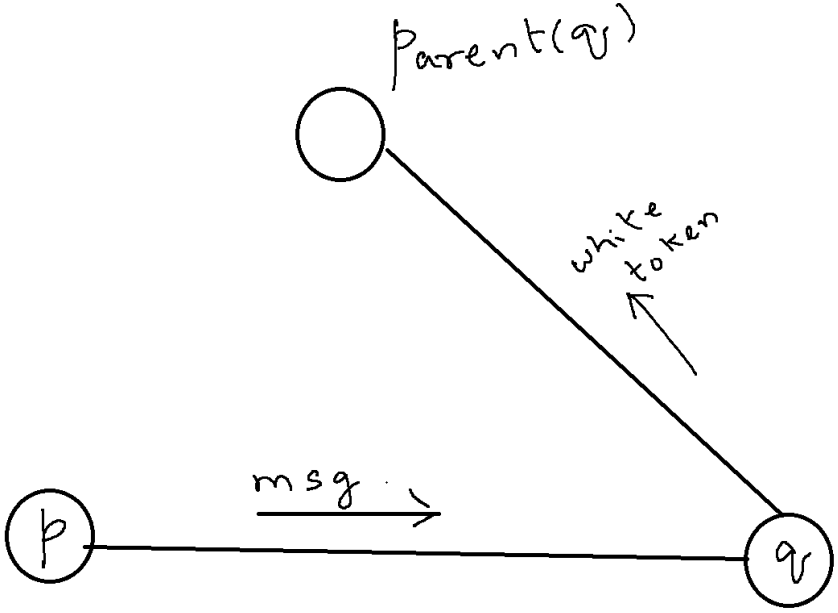
- The algorithm is quite expensive in the worst case.
- Essentially, does not optimize on when to kickstart the signals.
- Not every message is a trigger to restart termination detection.
- The next algorithm will fix some of these loopholes.

Message-Optimal Termination Detection

- Assumptions on the system are as follows
  - FIFO order of message delivery
  - A spanning tree of the processors and
  - A leader

The Previous Algorithm

- Consider the example shown.
- Suppose before node q receives message m, it has already sent a white token to its parent.
- Node p can not send a white token to its patent until node q becomes idle.
- To ensure this, node p changes its color to black and sends a black token to its parent so that termination detection is performed again.



To Optimize...

- If a process p that sends a message to a process q, then process p should wait until q becomes idle.
- To this end, process q can send an acknowledgement to process p indicating that the actions of q due to the message of p are complete.
  - This message helps p to send a white token to the parent of p.
- This may happen recursively too!
  - Process q could wake up an idle process r and will be waiting for an acknowledgement from r.
  - Only then q can send an acknowledgement to p.
- In general, a node will send a white token only after it has received an acknowledgement for every message it has sent and has received a white token from each of its children.

To Optimize

- Can develop this into an algorithm that is optimal in terms of the number of messages needed to detect termination.
- Maintain a stack of acknowledgements to be received.
- Read about the details from the book.

Other Variants

- Other variants work under the setting that a process that is idle may not become active on receipt of a message from just one other process.
- The transition from idle to active could be rule driven
  - A process P in the idle state turns active when it receives a message from processes Q1, Q2, and Q3.
  - A process P in the idle state turns active when it receives a message from processes Q1 or Q2 or Q3.
  - A process P in the idle state turns active when it receives a message from k out of n processes.
  - A process P has a predicate Φ that has to be true.